

PROJET OBSILOCK

Documentation Technique

Architecture · Sécurité · API · Base de données · Déploiement

Version	1.0
----------------	-----

Stack technique	Java 21 / JavaFX · PHP 8.2 Slim · MySQL 8 · Docker · LibSodium
------------------------	--

Audience	Développeurs, jury BTS SIO SLAM
-----------------	---------------------------------

1. Architecture globale du système

1.1 Vue d'ensemble

ObsiLock adopte une architecture trois-tiers découplée :

- Tier 1 (Présentation) : application desktop Java 21 / JavaFX
- Tier 2 (Logique métier) : API REST PHP 8.2 — Slim Framework
- Tier 3 (Données) : MySQL 8 + stockage physique des blobs chiffrés

Les tiers 2 et 3 sont conteneurisés via Docker. Le tier 1 (client) s'exécute localement sur le poste de l'utilisateur.

1.2 Flux de communication

- Le client JavaFX envoie des requêtes HTTP/REST via ApiClient.java
- Chaque requête authentifiée porte un header Authorization: Bearer <token_JWT>
- Le middleware PHP valide le token avant toute action
- Les réponses sont au format JSON (sauf téléchargement : binaire ou ZIP)

2. Backend — API REST PHP 8.2

2.1 Structure du projet backend

Chemin	Rôle
public/index.php	Point d'entrée — définition de toutes les routes de l'API
src/Controller/AuthController.php	Inscription, connexion, génération de token JWT
src/Controller/FileController.php	Upload, téléchargement, versionnement, suppression
src/Controller/FolderController.php	Création, liste, suppression de dossiers (récursif)
src/Controller/ShareController.php	Génération et consommation de liens de partage tokenisés
src/Middleware/JwtMiddleware.php	Validation du token JWT sur toutes les routes protégées
storage/user_{id}/	Stockage physique des blobs chiffrés par utilisateur

2.2 Routes de l'API

POST	/auth/register	Créer un compte utilisateur
POST	/auth/login	Authentifier et retourner un JWT
GET	/files/{id}	Télécharger un fichier (déchiffré)
POST	/files	Uploader un fichier (chiffré)
DELETE	/files/{id}	Supprimer un fichier et ses versions
GET	/files/{id}/versions	Lister les versions d'un fichier

GET	/folders	Lister les dossiers (racine ou enfants)
POST	/folders	Créer un dossier
GET	/folders/{id}/download	Télécharger un dossier en ZIP (déchiffré à la volée)
POST	/shares	Créer un partage tokenisé
GET	/shares/{token}	Accéder à une ressource partagée
GET	/user/quota	Consulter le quota (utilisé / total)

2.3 Authentification JWT

Le système utilise des tokens JWT (JSON Web Tokens) signés côté serveur :

- Génération : à la connexion, le serveur signe un payload {user_id, email, exp} avec une clé secrète HMAC-SHA256
- Transmission : le client inclut le token dans le header HTTP Authorization: Bearer <token>
- Validation : le middleware JwtMiddleware.php vérifie la signature et l'expiration à chaque requête protégée
- Durée de vie : 24 heures (configurable)

3. Architecture de sécurité — Chiffrement libsodium

3.1 Algorithme utilisé

Le projet utilise l'algorithme AEAD XChaCha20-Poly1305 via la bibliothèque libsodium (attention à ne pas confondre avec XSalsa20-Poly1305 — une variante plus ancienne de libsodium). XChaCha20-Poly1305 est le mode recommandé pour les nouveaux projets : il utilise un nonce étendu de 192 bits (contre 64 bits pour ChaCha20 standard), éliminant pratiquement tout risque de collision de nonce même avec une génération aléatoire. Cet algorithme garantit à la fois la confidentialité (chiffrement XChaCha20) et l'intégrité (authentification Poly1305).

3.2 Mécanisme Key Envelope

- Étape 1 — Génération DEK : à chaque upload, une clé DEK (Data Encryption Key) aléatoire unique est générée
- Étape 2 — Chiffrement du contenu : le fichier est chiffré avec la DEK + un nonce aléatoire
- Étape 3 — Key Envelope : la DEK est elle-même chiffrée par une clé maître KEK (Key Encryption Key) stockée uniquement côté serveur
- Étape 4 — Stockage : seuls le blob chiffré et les métadonnées (nonce, key_envelope, key_nonce, auth_tag) sont persistés en base

3.3 Métadonnées de chiffrement stockées

nonce	Valeur aléatoire unique utilisée lors du chiffrement du contenu (XChaCha20)
key_envelope	DEK chiffrée par la KEK serveur — permet de retrouver la DEK au déchiffrement

key_nonce	Nonce utilisé pour chiffrer la DEK (Key Envelope)
auth_tag	Tag d'authentification Poly1305 — garantit l'intégrité du contenu

4. Frontend — Application Java 21 / JavaFX

4.1 Structure du projet frontend

App.java	Point d'entrée JavaFX — gestion du thème global (applyTheme)
MainController.java	Logique de l'explorateur — upload, téléchargement, navigation dossiers
ApiClient.java	Client HTTP centralisé — tous les appels REST vers le backend
resources/*.fxml	Définition des vues (Scene Builder) — interface déclarative JavaFX
resources/style-javafx.css	Thème sombre — variables CSS JavaFX
resources/style-light.css	Thème clair — variables CSS JavaFX

4.3 ApiClient.java — Centralisation des appels HTTP

Tous les appels HTTP vers l'API sont centralisés dans ApiClient.java. Cette classe gère :

- L'injection automatique du header JWT sur les requêtes authentifiées
- La sérialisation / désérialisation JSON
- La gestion des codes d'erreur HTTP (401, 403, 404, 410, 507)
- Le téléchargement binaire (fichiers, ZIP)

5. Base de données MySQL 8

5.1 Schéma des tables

users	user_id (PK), email (UNIQUE), password, quota_total,	Mot de passe hashé bcrypt
	quota_used	
folders	folder_id (PK), user_id (FK), parent_id (FK nullable), name	Auto-référence pour la récursivité
files	file_id (PK), folder_id (FK), filename, size, checksum	checksum = SHA256 du fichier original
versions	version_id (PK), file_id (FK), blob_path, nonce, key_envelope, key_nonce, auth_tag, created_at	1 fichier → N versions

shares	share_id (PK), token (UNIQUE), file_id (FK nullable), folder_id (FK nullable), expires_at, max_uses, uses_count, label	Héritage : file_id XOR folder_id
--------	--	----------------------------------

5.2 ORM Medoo

Le backend utilise Medoo comme ORM léger pour PHP. Les requêtes sont construites via des tableaux PHP associatifs, évitant les injections SQL. Les opérations de quota sont effectuées en une seule transaction pour garantir la cohérence.

6. Déploiement Docker

6.1 Conteneurs

- Conteneur php-apache : image PHP 8.2 + Apache, expose le port 80/443
- Conteneur mysql : image MySQL 8, base de données obsilock, volume persistant
- Volume storage : montage du répertoire storage/ pour la persistance des blobs

6.2 Variables d'environnement clés

JWT_SECRET	Clé secrète de signature des tokens JWT
DB_HOST / DB_NAME / DB_USER / DB_PASS	Paramètres de connexion MySQL
KEK_SECRET	Clé maître de chiffrement des DEK (Key Encryption Key)
STORAGE_PATH	Chemin absolu du dossier de stockage des blobs

7. Tests

7.1 Tests fonctionnels API

Les routes API ont été testées via Postman et cURL. Les scénarios couverts :

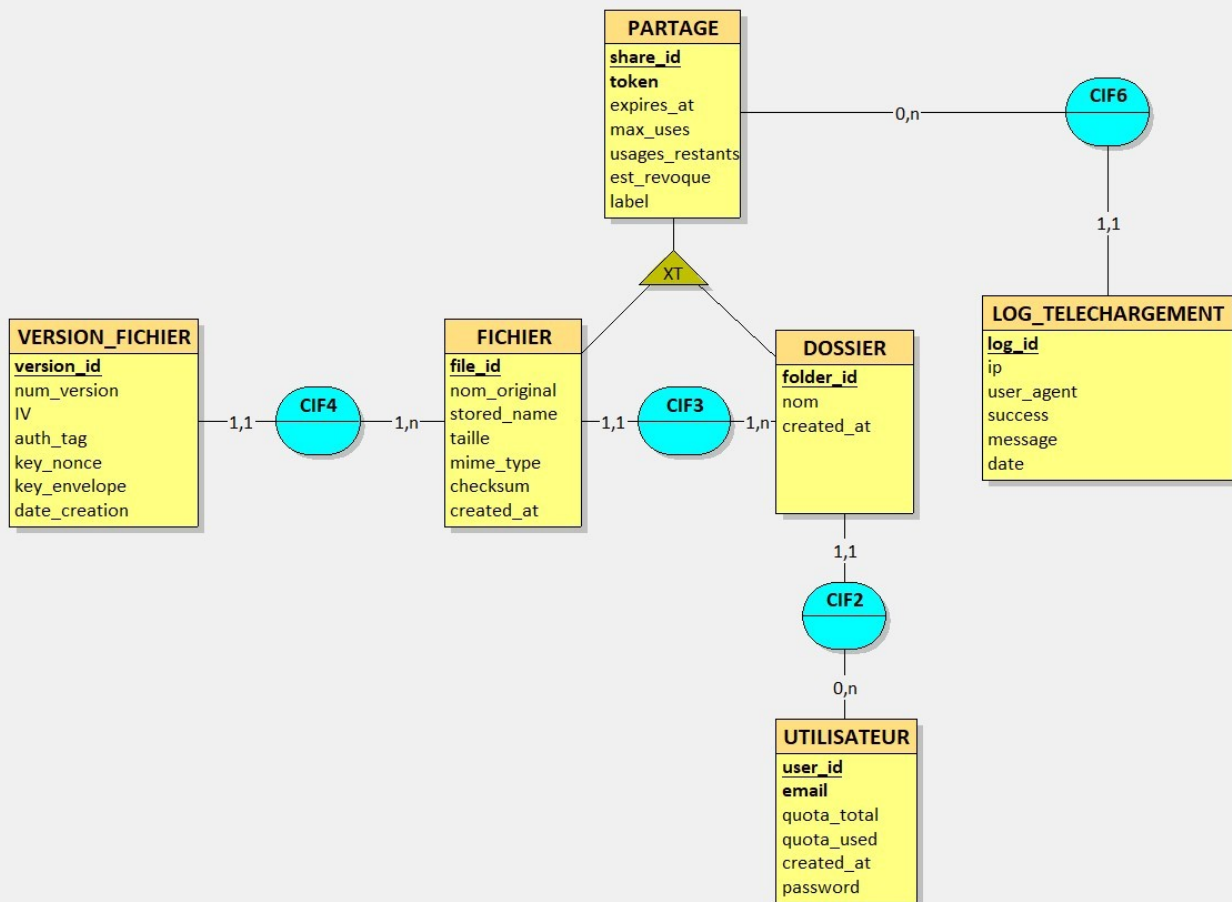
- Inscription / connexion : vérification des tokens JWT générés
- Upload : contrôle du quota avant et après, vérification du blob physique
- Téléchargement : comparaison checksum fichier original vs fichier restitué
- Partage expiré : vérification du retour 410 Gone après expiration
- Quota dépassé : vérification du retour 507 Insufficient Storage

7.2 Tests de sécurité

- Accès sans token : vérification du retour 401 Unauthorized
- Token altéré : vérification du rejet par le middleware JWT
- Tentative d'accès à un fichier d'un autre utilisateur : vérification du 403 Forbidden

8. Schéma Relationnel

8.1 MCD (Modèle Conceptuel de Données)



8.2 Diagramme de cas UML (Unified Modeling Language)

